

Python intro: teaching map

WEEK 2 / 1 OF 5 / Setup and object types / Suggested time: 12 minutes

Companion to **intro_full.ipynb**. Follow the notebook from top to bottom. Timings are a suggested 50-minute teaching plan, including short exercises.

Before students arrive

Open the notebook with the intended Python kernel. Use **sessions/week_2/scripts/intro** as the working directory. Confirm NumPy is installed, **my_library.py** is beside the notebook, and **../data/cities.txt** exists. The file exercise writes **cities_clean.txt**.

Teaching rhythm: Ask students to predict a result, run the cell, then explain what changed. Re-running cells can change existing state; restart and run in order for a clean demonstration.

Notebook basics

Markdown cells explain; code cells execute Python. A **#** inside a code cell starts a Python comment. Demonstrate running with Shift+Enter and creating/changing cells using the notebook toolbar. Keyboard bindings can vary; check them locally before class.

Topic 1 | Names, values, types

```
is_student = True           # bool
population_ch = 8_900_000    # int
inflation = 1.6              # float
city = "Zürich"              # str
cantons = ["ZH", "BE", "LU", "UR"] # list
coords_uni_zh = (47.376, 8.548) # tuple
gdp = {"CH": 800, "DE": 4500, "LT": 70} # dict
```

Say: A variable name refers to a value. `type(value)` tells us its type. A list is mutable; a tuple does not allow replacing its items. A dictionary looks up values by keys.

Demonstrate: `cantons.append("VD")` changes the existing list. An f-string inserts values into text: `f"Welcome to {city}!"` becomes `Welcome to Zürich!`

Ask: What does `int("2025")` produce? **2025, an int**. What about `float("3.14")`? **3.14, a float**. `int("3.14")` raises `ValueError`.

Mini-exercise answer | 2 minutes

```
major = "Economics"
ects = 6
courses = {"ECON101": 6, "PROG101": 3}
print(type(major), type(ects), type(courses))
# <class 'str'> <class 'int'> <class 'dict'>
```

Indexing and slicing

WEEK 2 / 2 OF 5 / Topic 2 / Suggested time: 8 minutes

Opening question: How do we retrieve one value from a collection? Sequences use positions; dictionaries use keys.

Live walkthrough

```
sbb_line = "IC5: Zürich HB → Lausanne"
sbb_line[0]      # "I"
sbb_line[-1]     # "e"
sbb_line[4:10]   # " Zürich" (space plus five letters)
sbb_line[::-1]   # reversed string
```

```
fares = [3.4, 4.2, 8.8, 12.5]
fares[0]      # 3.4
fares[1:3]    # [4.2, 8.8]
fares[-1]     # 12.5
coords_uni_zh[0] # 47.376
```

Say: Positions start at zero. The stop position is excluded: [1:3] selects indices 1 and 2. Negative indices count from the end. A slice returns a sequence, even if it contains only one item.

Board sketch: For fares, write indices 0, 1, 2, 3 above the four values and -4, -3, -2, -1 below them.

Dictionary access

```
gdp["CH"]      # 800 initially
gdp["CH"] = gdp["CH"] * 1.01 # 808.0 after one run
gdp.get("FR", "No data")    # "No data"
```

Watch for: gdp[0] asks for the key 0, not the first entry. A missing key with square brackets raises KeyError. .get() can provide a fallback.

Notebook state: Running the update again multiplies the updated value by 1.01 again. Re-run the definition of gdp to reset it.

Mini-exercise answer | 2 minutes

```
name = "University of Zurich"
name[14:]      # "Zurich"
name[-6:]     # "Zurich" - another valid answer
cantons[-2:]  # ["UR", "VD"] after the append in Topic 1
```

Check understanding: How do fares[1] and fares[1:2] differ? **4.2** is a float; **[4.2]** is a list.

Common mistake: A single out-of-range index raises IndexError; slices can extend beyond the end without raising that error.

Operations and decisions

WEEK 2 / 3 OF 5 / Topics 3-4 / Suggested time: 12 minutes

Topic 3 | Calculate, transform, inspect

```
monthly_ticket = 70
months = 6
cost_semester = monthly_ticket * months # 420
msg = "UZH" + " Economics"
msg.upper() # "UZH ECONOMICS"
prices = [12.5, 3.4, 8.8, 4.2]
len(prices) # 4
sorted(prices) # [3.4, 4.2, 8.8, 12.5]
prices.append(6.9)
```

Say: sorted(prices) returns a new sorted list. append changes prices in place and returns None. After the append, prices is [12.5, 3.4, 8.8, 4.2, 6.9]. String methods return new strings.

Quick arithmetic: $7 / 2 = 3.5$; $7 // 2 = 3$; $7 \% 2 = 1$; $2 ** 3 = 8$. With strings, + joins and * repeats. append adds one item; extend adds each item from an iterable.

Mini-exercise answer | 2 minutes

```
cities = ["Zürich", "Bern", "Basel"]
print(len(cities)) # 3
print(sorted(cities)) # ["Basel", "Bern", "Zürich"]
```

Topic 4 | Boolean expressions select a branch

```
age = 23
has_halbtax = True
eligible = (age < 25) or has_halbtax # True

income = 2200
if income < 2000:
    bracket = "low"
elif income < 5000:
    bracket = "middle"
else:
    bracket = "high"
print(bracket) # middle
```

Ask: At age 26 without Halbtax? **False**. At age 26 with Halbtax? **True**. With and instead of or, both conditions would need to be true. This is a toy policy, not a real fare rule.

Boundary check: Income 1999 gives low; 2000 gives middle; 4999 gives middle; 5000 gives high. Only the first matching branch runs.

Watch for: = assigns, == compares. Conditions need a colon; blocks need consistent indentation. True and False are capitalized.

Read, clean, write a file

WEEK 2 / 4 OF 5 / Topic 5 / Suggested time: 10 minutes

Explain the pipeline before running

Path → **text** → **lines** → **cleaned list** → **output file**. Pause here to introduce for and continue: the notebook uses them before a dedicated loop lesson.

```
from pathlib import Path
DATA = Path(".././data")
in_path = DATA / "cities.txt"
out_path = DATA / "cities_clean.txt"

raw_text = in_path.read_text(encoding="utf-8")
lines = raw_text.splitlines()
cleaned = []
for ln in lines:
    ln = ln.strip()
    if not ln:
        continue
    cleaned.append(ln.title())

out_path.write_text("\n".join(cleaned), encoding="utf-8")
```

Walk through one line: " geneva " becomes "geneva" after strip(), then "Geneva" after title(). The empty line is skipped because an empty string is false in a condition.

Step	What to explain
Path(".././data")	Go up two folders from the working directory, then into data. / joins path components.
splitlines()	Split text at line boundaries; remove the line separators.
for ln in lines	Visit each line in order. continue skips the rest of the current iteration.
"\n".join(cleaned)	Combine the cleaned names with a newline between them.
write_text(...)	Write UTF-8 text; replace the output file if it already exists.

Expected result with the supplied file

The input has **6 lines**, including one blank line. The output has **5 entries**: Zurich, Geneva, Basel, Lausanne, Bern. The preview shows all five because cleaned[:5] selects up to five items.

Be precise: strip() removes outer whitespace; it does not collapse repeated spaces inside a name. title() changes letter case; it does not add accents, so zurich becomes Zurich.

If the file is not found: Inspect Path.cwd() and in_path.resolve(). Run from the intro folder or adjust the path to match the actual working directory.

Imports and final checks

WEEK 2 / 5 OF 5 / Topic 6 / Suggested time: 8 minutes

Libraries provide reusable code

```
import numpy as np
import math
prices = [3.4, 4.2, 8.8, 12.5]
print(np.mean(prices)) # 7.225
print(math.sqrt(81))   # 9.0
```

Say: import makes a module available. as np gives NumPy a shorter name. The dot selects something inside the module. math is included with Python; NumPy is an additional package.

If NumPy is missing: Use the notebook's suggested %pip install numpy in a separate cell, then retry the import in the intended kernel. This needs package access and may need a kernel restart.

The local module | my_library.py

```
city = "Zürich"
currency = "CHF"
ticket_price = 4.20

def total_cost(number_of_tickets):
    return round(number_of_tickets * ticket_price, 2)
```

A .py file can be a module. def defines a reusable function; its body runs when called. return sends the calculated value back to the caller.

```
from my_library import city, ticket_price
print(city)                # Zürich
print(ticket_price)         # 4.2

import my_library
print(my_library.currency)  # CHF
print(my_library.total_cost(3)) # 12.6
```

Ask: What does the 3 mean? **It becomes number_of_tickets inside the function.** Why is 4.20 displayed as 4.2? **A float does not retain display formatting.**

Watch for: Omit .py in an import. The local file must be discoverable; use the intro working directory. Python normally runs module top-level code once on first import per kernel session. After editing a module, restart the kernel and run again to avoid stale definitions.

Exit check | ask students without running

1. What is the difference between a list and a dictionary? **Position-based access versus key-based access.**
2. What does [1:3] select? **Indices 1 and 2.**
3. What changes after append? **The existing list.**
4. What does continue do? **Move to the next loop iteration.**
5. Why import code? **To reuse definitions and functionality.**

Source: intro_full.ipynb, my_library.py and ../../data/cities.txt in the Week 2 materials. This guide follows the completed notebook; teaching prompts and timing are additions.